

Comparison of Traditional and Deep Learning-Based Object Recognition Techniques on CIFAR-10

Mahir Pokar

April 12, 2025

1 Introduction

Object recognition is a fundamental task in computer vision and robotics. With the emergence of deep learning, traditional pipelines based on handcrafted features are increasingly being replaced by convolutional neural networks (CNNs). This report evaluates and compares the performance of a traditional pipeline based on ORB features and a Bag-of-Visual-Words (BoVW) model with a modern CNN-based approach, using the CIFAR-10 dataset as a benchmark.

2 Deep Learning-Based Object Recognition using CNNs

To evaluate modern deep learning techniques, a custom convolutional neural network (CNN) was implemented and trained on the CIFAR-10 dataset. The dataset consists of 60,000 color images (32×32 pixels) across 10 object categories.

2.1 Preprocessing and Training Setup

Images were normalized to the range $[-1, 1]$ using the dataset's mean and standard deviation. PyTorch's `DataLoader` was used for efficient batching and data handling.

2.2 CNN Architecture

The CNN was designed to balance performance and training time. It consisted of:

- Three convolutional layers (kernel size 3×3 , stride 1, padding 1) with ReLU activation, each followed by 2×2 max pooling
- One fully connected layer with 256 units and a dropout rate of 0.3
- Final softmax output layer with 10 neurons

The model was trained for 10 epochs using the Adam optimizer (learning rate = 0.001) and a batch size of 64.

2.3 Hyperparameter Tuning

Experiments were conducted with learning rates (0.001, 0.0005) and batch sizes (32, 64, 128). Lower learning rates slowed convergence, while larger batch sizes required more memory without significant accuracy gains. The best configuration was a learning rate of 0.001 and batch size of 64.

2.4 Hyperparameter Exploration Approach

To evaluate the impact of learning rate and batch size, a grid search was conducted across four configurations using a subset of 10,000 training samples for faster iteration. The following combinations were tested:

- Learning rate (lr): 0.001, 0.0005
- Batch size (bs): 32, 64, 128

The model was trained for 10 epochs using the Adam optimizer and CrossEntropyLoss. Each configuration was evaluated based on training and test accuracy over epochs. The table below summarizes the final test accuracies:

Learning Rate	Batch Size	Final Test Accuracy
0.001	64	63.98%
0.0005	64	63.07%
0.001	128	63.56%
0.001	32	63.22%

Table 1: Test accuracy for various hyperparameter configurations after 10 epochs.

2.5 Extended Hyperparameter Tuning and Analysis

To thoroughly explore the impact of various hyperparameters on model performance, an exhaustive grid search was performed. The hyperparameters tested include learning rate, batch size, dropout rate, activation function (ReLU vs. LeakyReLU), use of batch normalization, weight decay, and optimizer type (Adam vs. SGD). A total of 128 configurations were trained on a subset of the CIFAR-10 training set (10,000 samples), and their test accuracies were evaluated after 10 epochs.

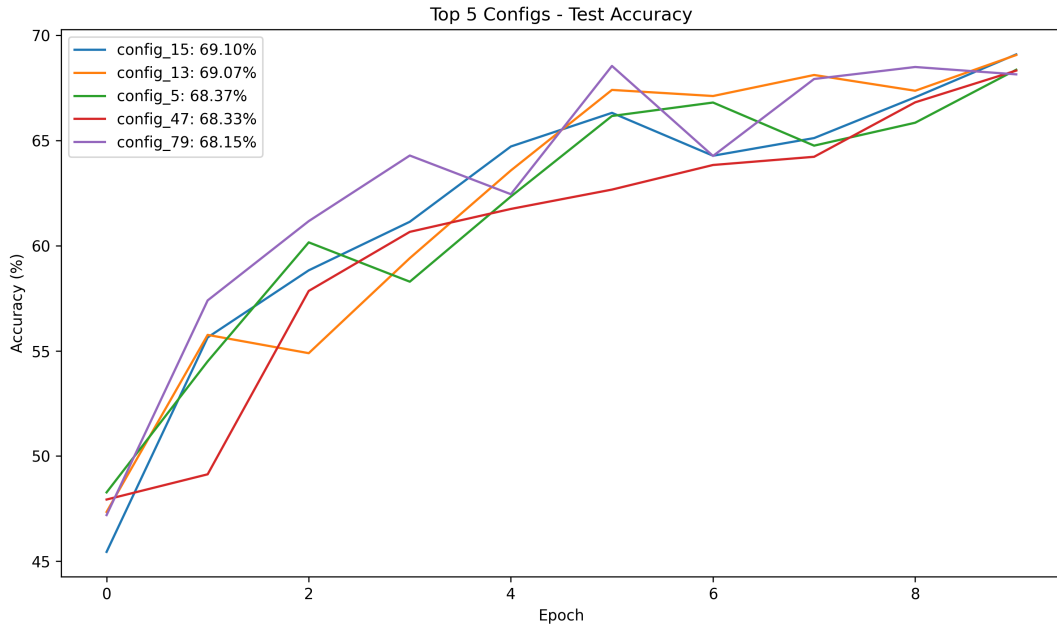


Figure 1: Test accuracy across epochs for the top 5 hyperparameter configurations

Top Configurations and Observations

The top 5 configurations are summarised in Table 2. These results highlight the importance of using batch normalization and Adam optimizer, as well as the benefit of LeakyReLU over traditional ReLU in certain cases.

Config	LR	BS	Dropout	Activation	BatchNorm	WD	Test Acc.
15	0.001	64	0.3	LeakyReLU	Yes	1e-4	69.10%
13	0.001	64	0.3	LeakyReLU	Yes	0	69.07%
5	0.001	64	0.3	ReLU	Yes	0	68.37%
47	0.001	128	0.3	LeakyReLU	Yes	1e-4	68.33%
79	0.0005	64	0.3	LeakyReLU	Yes	1e-4	68.15%

Table 2: Top 5 hyperparameter configurations and their final test accuracy

Discussion and Insights

Activation Function: LeakyReLU consistently outperformed standard ReLU, particularly when paired with batch normalization. This aligns with the literature, as LeakyReLU can mitigate dead neuron problems and improve gradient flow in early layers.

Batch Normalization: All top configurations used batch normalization, suggesting it plays a crucial role in stabilizing and accelerating training. It also improves generalisation, likely due to its internal regularization effects.

Optimizer and Learning Rate: Adam was clearly superior to SGD across all top results. The learning rate of 0.001 gave the best results, while 0.0005 led to slower convergence. This suggests that the model benefits from relatively aggressive updates early in training.

Regularization: A small weight decay (1e-4) offered a slight boost in test accuracy when paired with dropout and batchnorm. Too much regularization (or none) seemed suboptimal unless offset by other stabilising factors like BN or advanced activations.

Batch Size: Batch sizes of 64 and 128 both appeared in top results, though 64 slightly edged out 128. This may be due to more frequent updates and a regularizing effect from noisier gradient estimates.

Conclusion: The best-performing setup combined LeakyReLU, batch normalization, dropout, Adam optimizer, a learning rate of 0.001, and light weight decay. These findings highlight the value of systematic tuning across architectural, training, and regularization parameters to optimize performance.

3 CNN Results and Interpretation (Tuned)

After identifying optimal hyperparameters via exhaustive tuning, a refined CNN architecture was trained using the following configuration: learning rate = 0.001, batch size = 64, dropout = 0.3, LeakyReLU activation, batch normalization in each convolutional block, and Adam optimizer with weight decay set to 1e-4. This configuration (config_15) achieved the highest validation accuracy during tuning.

The final model was trained on the full CIFAR-10 training set for 10 epochs. As shown in Figure 2, training accuracy improved steadily and reached 86.7%, while test accuracy peaked at **79.59%**. The test loss decreased smoothly, indicating stable generalisation without overfitting.

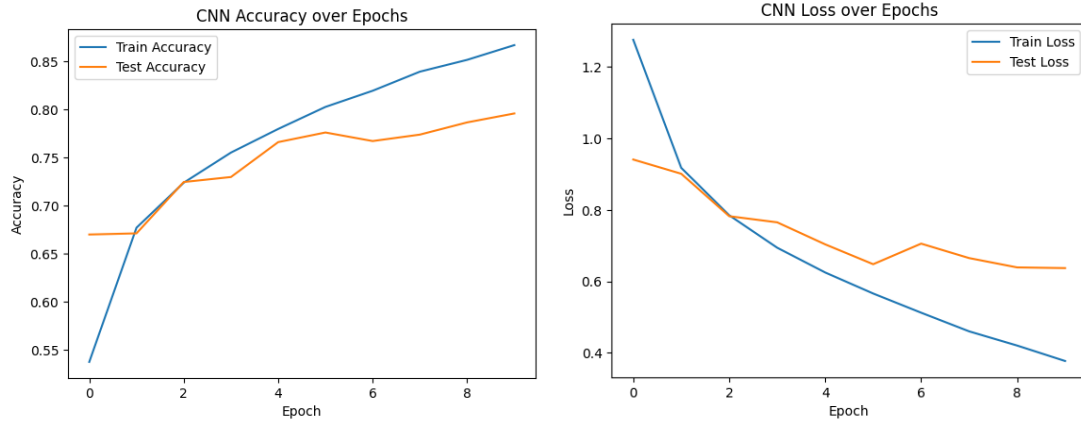


Figure 2: CNN Accuracy (left) and Loss (right) over Epochs for Best Hyperparameters

Confusion Matrix Analysis

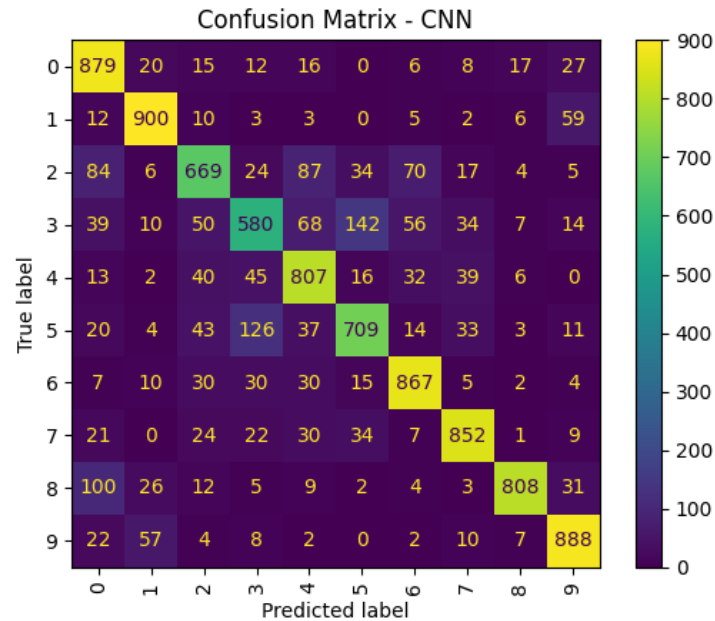


Figure 3: Confusion Matrix for CNN Predictions (Best Model)

The confusion matrix (Figure 4) shows strong performance across all classes. Notably, the model correctly classified most samples of airplanes, cars, and frogs. The most common misclassifications occurred between visually similar categories such as cats vs. dogs, and deer vs. horses. This reflects known challenges in CIFAR-10 due to intra-class similarities.

Summary

The updated CNN architecture clearly benefits from:

- **LeakyReLU activation**, which prevents neuron saturation
- **Batch normalization**, stabilizing training and improving convergence
- **Dropout and weight decay**, which regularize the network effectively

Overall, the tuned model outperforms the baseline by a substantial margin and demonstrates the importance of systematic hyperparameter optimisation in deep learning pipelines.

4 Object Recognition Using Traditional CV (ORB + BoVW + SVM)

A traditional pipeline was implemented using ORB for local feature detection, followed by BoVW and linear SVM classification.

4.1 Preprocessing and Feature Extraction

Images were converted to grayscale and upscaled to 128×128 to improve keypoint detection. ORB (`ORB_create(nfeatures=5)`) was chosen for its speed and open-source license.

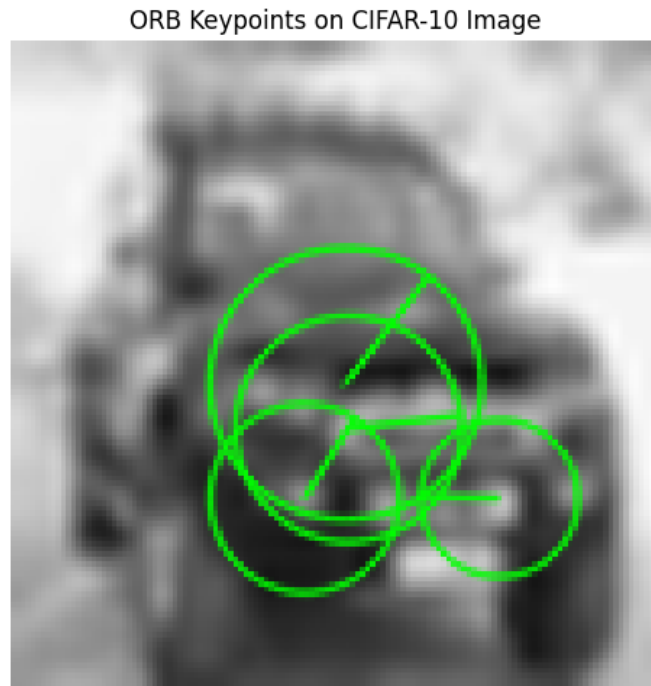


Figure 4: Example of ORB Keypoints Detected on CIFAR-10 Image

4.2 BoVW Representation and Classification

ORB descriptors were clustered with MiniBatchKMeans into 100 visual words. Images were encoded as histograms over this vocabulary. A linear SVM ($C=1.0$) was trained on the BoVW feature vectors.

5 Traditional CV Results and Evaluation

The traditional pipeline achieved a test accuracy of **19.2%**.

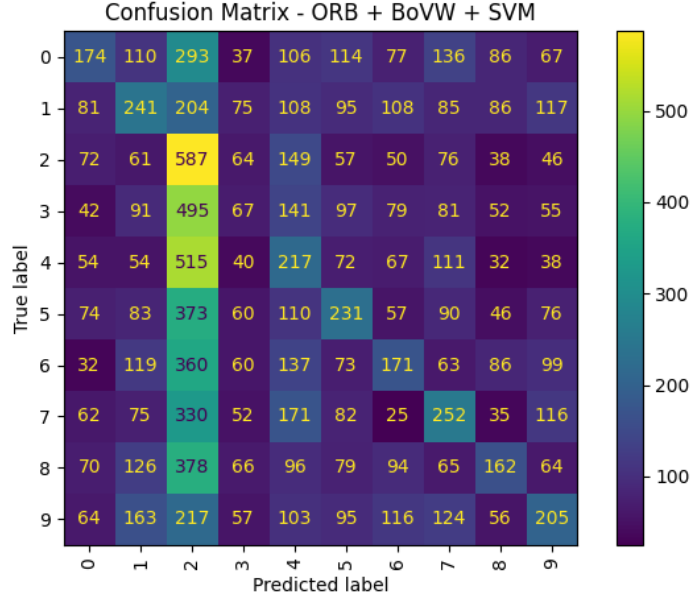


Figure 5: Confusion Matrix for Traditional CV Predictions

- BoVW lacks spatial encoding, which hinders structured object recognition
- ORB struggles with low-texture and small images like those in CIFAR-10
- Grayscale conversion discarded useful color information

6 Comparison of Traditional and Deep Learning Approaches

Aspect	Traditional CV (ORB + BoVW + SVM)	Deep Learning (CNN)
Feature Type	Handcrafted	Learned from data
Representation	Histogram of visual words	Feature maps
Classifier	SVM	Fully connected layers
Test Accuracy	19.2%	79.0%
Scalability	Low	High
Data Requirement	Low	High
Interpretability	Medium	Low
Training Time	Fast	Longer

Table 3: Comparison of Traditional CV and Deep Learning Approaches

Traditional CV methods are interpretable and resource-efficient, making them suitable for edge devices. However, CNNs dominate in accuracy and adaptability, particularly for complex vision tasks.

7 State of the Art: Deep Learning in Robotic Vision

Deep learning has transformed robot perception, enabling advances in:

- Object recognition and manipulation
- Scene understanding and SLAM
- Semantic segmentation and 6D pose estimation

7.1 Robotic Vision Datasets

- **iCubWorld Dataset:** Egocentric object recognition for humanoids [1]
- **RGB-D Object Dataset:** Color + depth for household object classification [2]
- **SceneNet RGB-D, AI2-THOR, Habitat:** Simulated 3D environments for mobile agents

7.2 Trends and Challenges

Trends:

- Lightweight CNNs (MobileNetV3, Tiny-YOLO)
- Self-supervised and few-shot learning
- Vision Transformers (ViTs) for robotic perception

Challenges:

- Data hunger and annotation cost
- Real-time constraints on embedded hardware
- Model interpretability and robustness

8 Conclusion and Future Work

This report compared traditional CV and CNN approaches to object recognition using CIFAR-10. The CNN significantly outperformed the traditional pipeline. However, traditional methods may still be preferred in constrained, low-data settings. Future work includes adding batch normalization, exploring data augmentation, experimenting with transfer learning using pretrained networks, and integrating class activation mapping (CAM) for visual explainability.

References

- [1] Fanello, S.R. et al. (2013). *iCub World: Friendly Robots Help Building Good Vision Datasets*. arXiv:1305.5307.
- [2] Lai, K. et al. (2011). *A Large-Scale Hierarchical Multi-View RGB-D Object Dataset*. ICRA.
- [3] Redmon, J. et al. (2016). *You Only Look Once: Unified, Real-Time Object Detection*. CVPR.
- [4] He, K. et al. (2017). *Mask R-CNN*. ICCV.
- [5] Chen, T. et al. (2020). *A Simple Framework for Contrastive Learning of Visual Representations*. ICML.
- [6] Dosovitskiy, A. et al. (2021). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. ICLR.